UNIT-II — SYNCHRONIZATION (Complete Notes)

Labels: 📘 official syllabus · 📝 assignment-emphasized · ➕ additional/exam-oriented

---

1. INTRODUCTION TO SYNCHRONIZATION 📘📝 (A2-1a)

Definition

Synchronization in a distributed system means **coordinating the actions of independent processes** so that they agree on **time/order of events, access shared resources safely, and reach common decisions**, even though there is **no shared memory and no global clock**.

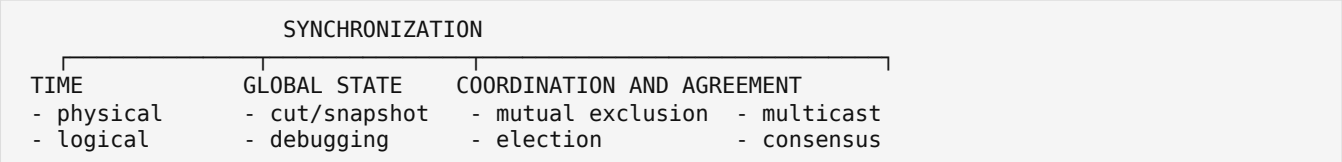
Simple (🗣️)

Alag-alag computers ko ek "team" ki tarah chalana: kaun pehle, kaun baad me, kaun resource use karega, sab kis baat pe agree honge – ye sab synchronization hai.

Why is it needed? (Problems caused by distribution)

Problem	Reason	Solution in this unit
No global clock	each node has own clock, drift	Physical clock sync (Cristian/Berkeley/NTP), Logical clocks
Event ordering	messages have delay	Happened-before, Lamport/Vector clocks
No global state	no node sees whole system at once	Global states, snapshot
Concurrent access to a resource	many processes want the same resource	Distributed mutual exclusion
Need for a leader	someone must coordinate	Election algorithms
Group communication	send to many reliably/ordered	Multicast
Agreement despite faults	processes may crash/lie	Consensus

Map of Unit-II



Exam points

- Answer to "Explain synchronization" (2 marks): definition + 3 reasons (no global clock, concurrency, independent failures) + mention events/process states.

🧠 Remember This

> Synchronization = agree on **time/order**, **shared resources**, **leader**, **state**, **decision** — without shared memory or clock.

---

2. TIME AND GLOBAL STATES – INTRODUCTION 📘

- Each computer has a **hardware clock** (oscillator + counter) → software clock $C(t) = \alpha \cdot H(t) + \beta$.
- Time is needed for: **timestamps** (e-commerce, banking), **auditing/logging**, **authentication (Kerberos)**, **consistency (make)**, **ordering events**.
- Since perfect synchronization is impossible, DS uses (a) **Synchronized physical clocks** (approximate real time), (b) **Logical clocks** (only order matters), (c) **Global state** notion (what is the state "at some time").
- **Key idea:** In DS we often care only about **order of events (causality)**, not exact real time.

```
Real time t ──────────────────────────────►
Process P1 clock: 10:00:00 10:00:03 ...
Process P2 clock: 10:00:02 10:00:05 ...   ← different! (skew)
```

🧠 Remember This

> Time can be **physical** (real, needs sync) or **logical** (order only).

---

3. EVENTS AND PROCESS STATES 📘📝 (A2-1a)

Definitions

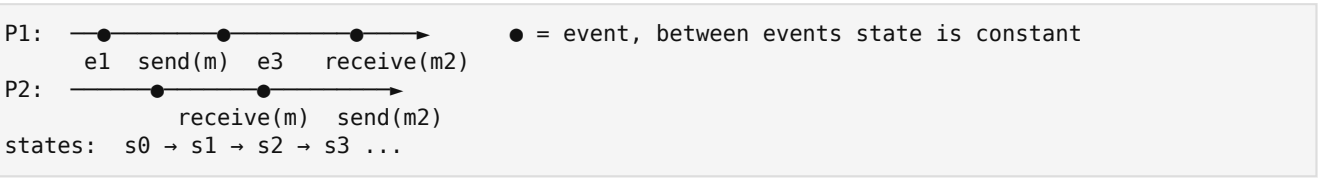
- **Process:** sequence of actions; a distributed system = set of processes p1..pN communicating **only** by messages.
- **Process state:** the values of all variables of the process (registers, memory, program counter, etc.) at a given moment.
- **Event:** an **action** that changes the process state. Three kinds: 1. **Internal (local) event** – computation changes local variable. 2. **Send event** – process sends a message. 3. **Receive event** – process receives a message.
- **History of a process:** ordered sequence of its events: `history(pi) = <e0, e1, e2, …>` (events totally ordered within one process).

- **Local state s_i^k:** state of pi just before event e_k (i.e., after k events).

Simple (🗣️)

Process ek state machine hai: har event (compute/send/receive) use ek state se dusri state me le jata hai.

Timeline diagram



Role of events and process states in synchronization

1. **Ordering events** – within a process events are totally ordered; across processes only via messages (send happens before receive).
2. **Global state** = collection of process states + channel states; consistency depends on which events have occurred.
3. **Detecting properties** (deadlock, termination) & **debugging** need to know sequences of states.
4. **Logical clocks** timestamp events; **state recording** takes snapshots of states.

Exam points

- Draw a timeline with two processes and one message; label send/receive; state clearly "events change process state".

🧠 Remember This

> State = data of process; **event** (internal/send/receive) changes the state; history = ordered list of events.

---

4. HAPPENED-BEFORE RELATION (→) 📘📝 (A2-1b)

Definition (Lamport, 1978)

The **happened-before** relation `a → b` is the **smallest relation** satisfying: 1. **HB1 (same process):** if a and b are events in the same process and a occurs before b, then `a → b`. 2. **HB2 (message):** if a = **send(m)** and b = **receive(m)**, then `a → b`. 3. **HB3 (transitivity):** if `a → b` and `b → c`, then `a → c`.

Concurrent events: if neither `a → b` nor `b → a`, then `a || b` (concurrent).

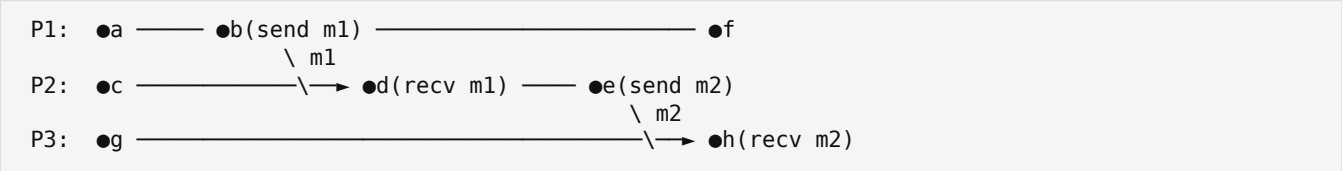
Simple (🗣️)

`a → b` ka matlab: a **cause kar sakta hai** b ko (ya a pehle ho chuka tha jiski khabar b tak pahunch sakti hai). Agar koi information-path nahi, to events **concurrent** hain – unka order decide nahi ho sakta (aur zarurat bhi nahi).

Properties

- **Irreflexive** (a ↛ a), **transitive**, **antisymmetric** → a **partial order** (not total).
- It captures **potential causality**, not real time.

Example



- a → b (same process), b → d (message m1), d → e, e → h (message m2).
- By transitivity: a → d, b → e, **b → h**, a → h.
- **Concurrent:** c ∥ a? c is in P2 before d; nothing links a and c → **a ∥ c**. Also f ∥ d (no path).

Exam points

- Always list the 3 rules. State: "→ gives partial order, so some events are concurrent".

🧠 Remember This

> HB: same-process order + send→receive + transitivity. No path = concurrent (∥).

---

5. LOGICAL CLOCKS — LAMPORT LOGICAL CLOCK 📘📝 (A2-1b)

5.1 Why logical clocks?

Physical clocks can't be perfectly synchronised, and we usually care about **order** not real time. A **logical clock** assigns **numbers (timestamps)** to events in a way that respects causality.

Definition

A **Lamport logical clock** is a **monotonically increasing software counter** `L_i` at each process `p_i`, used to timestamp events such that: **if a → b then L(a) < L(b)** (clock condition).

5.2 Rules (Lamport's implementation rules)

- **LC1 (before every event):** `L_i := L_i + 1` (increment before executing each local or send event).
- **LC2 (send):** when `p_i` sends message m, it first applies LC1, then attaches `t = L_i` to m.
- **LC3 (receive):** when `p_j` receives (m, t): `L_j := max(L_j, t)`, then apply LC1: `L_j := L_j + 1`, then timestamp the receive event.

(Equivalent common form: on receive, `L_j = max(L_j, t) + 1`.)

5.3 Algorithm (pseudocode)

```
Initially:   L_i = 0

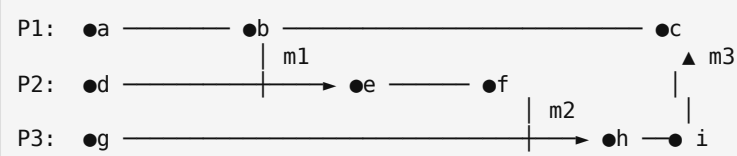
On internal event:      L_i = L_i + 1
On send(m) to p_j:      L_i = L_i + 1
                        send (m, L_i)
On receive (m, t):      L_i = max(L_i, t) + 1
```

5.4 Properties

- If `a → b` then `L(a) < L(b)`.
- **Converse is NOT true:** `L(a) < L(b)` does **not** imply `a → b` (they might be concurrent).
- Cannot detect concurrency. (Vector clocks can.)
- **Total ordering** (➕): break ties by process id: (L(a), pid(a)) < (L(b), pid(b)). Used in mutual exclusion (Lamport / Ricart-Agrawala).

5.5 Worked example 1 – step-by-step

Events: P1: a, b (send m1 to P2), c (receive m3 from P3). P2: d, e (receive m1), f (send m2 to P3). P3: g, h (receive m2), i (send m3 to P1).



Calculation (all clocks start at 0): | Event | Process | Type | Calculation | L | |---|---|---|---|---| | a | P1 | internal | 0+1 | **1** | | b | P1 | send m1 | 1+1 | **2** (m1 carries 2) | | d | P2 | internal | 0+1 | **1** | | e | P2 | receive m1 (t=2) | max(1,2)+1 | **3** | | f | P2 | send m2 | 3+1 | **4** (m2 carries 4) | | g | P3 | internal | 0+1 | **1** | | h | P3 | receive m2 (t=4) | max(1,4)+1 | **5** | | i | P3 | send m3 | 5+1 | **6** (m3 carries 6) | | c | P1 | receive m3 (t=6) | max(2,6)+1 | **7** |

Timeline with timestamps:



Observations: `b → e` and 2 < 3 ✓. `a` (1) and `d` (1) are concurrent and have equal timestamps → break tie with process id. `d(1) < b(2)` but d ∥ b → shows **converse false**.

5.6 Exam-style numerical questions (with answers)

Q1. Three processes P1, P2, P3 start with clock 0. P1 executes event a, then sends m to P2 (event b). P2 executes event c, then receives m (event d), then sends m' to P3 (event e). P3 executes f, then receives m' (event g). Find all Lamport timestamps. **Ans:** a=1, b=2 (m carries 2). P2: c=1, d=max(1,2)+1=3, e=4 (m' carries 4). P3: f=1, g=max(1,4)+1=5.

Q2. P1's clock is 10 and receives a message with timestamp 15. New value? **Ans:** max(10,15)+1 = **16**.

Q3. P2's clock is 20 and receives message with timestamp 12. **Ans:** max(20,12)+1 = **21**.

Q4. P1: events at clock 1,2,3 (3 = send to P2). P2 has local clock values 1,2 then receives message (ts 3), sends to P3, P3 receives. If P2's clock before receive = 2 and P3's clock before receive = 0, find receive timestamps. **Ans:** P2 receive = max(2,3)+1 = 4; P2 send = 5; P3 receive = max(0,5)+1 = 6.

Q5 (concurrency). Event x in P1 has L=5, event y in P2 has L=6, and no messages are exchanged. Can we say x → y? **Ans:** No. Lamport clocks only guarantee a→b ⇒ L(a)<L(b). x and y are concurrent.

Q6 (increment d=1). P1 clock 0: internal, internal, send; P2 clock 0: internal, receive (from P1's send). **Ans:** P1: 1,2,3 (send ts=3). P2: 1, then max(1,3)+1 = 4.

5.7 Advantages / limitations

Advantages	Limitations
Simple, small overhead (one integer)	Cannot detect concurrency
Preserves causality (a→b ⇒ L(a)<L(b))	L(a)<L(b) does not imply a→b
No physical clock needed	Timestamps are not real time
Basis for total ordering & mutual exclusion	Ties must be resolved with process id

🧠 Remember This

> Increment before every event; attach timestamp when sending; on receive: **max(local, msg) + 1**. a→b ⇒ L(a)<L(b) (not vice versa).

---

6. LOGICAL TIME – VECTOR CLOCKS ➕ (extension of logical clocks, often asked)

Definition

A **vector clock** at process p_i (system of N processes) is an **array V_i[1..N]** of integers. `V_i[i]` counts events at p_i; `V_i[j]` = p_i's knowledge of p_j's latest event count.

Rules

1. Initially V_i[j] = 0 for all j.
2. **Before each event** at p_i: `V_i[i] := V_i[i] + 1`.
3. **Send:** after step 2, attach the whole vector `t = V_i` to message.
4. **Receive (m, t):** `V_i[j] := max(V_i[j], t[j])` for all j; then `V_i[i] := V_i[i] + 1`.

Comparison of vectors

- `V = V'` iff all components equal.
- `V ≤ V'` iff **V[j] ≤ V'[j] for all j**.
- `V < V'` iff V ≤ V' and V ≠ V'.
- **a → b ⇔ V(a) < V(b)** (both directions!).
- **a ∥ b** iff neither V(a) < V(b) nor V(b) < V(a).

Example (same events as §5.5 with vector clocks, 3 processes)

Event	Calculation	Vector
a (P1 internal)		(1,0,0)
b (P1 send m1)		(2,0,0)
d (P2 internal)		(0,1,0)
e (P2 receive m1)	max((0,1,0),(2,0,0)) = (2,1,0) → +1 own	(2,2,0)
f (P2 send m2)	own+1	(2,3,0)
g (P3 internal)		(0,0,1)
h (P3 receive m2)	max((0,0,1),(2,3,0))=(2,3,1) → +1 own	(2,3,2)
i (P3 send m3)		(2,3,3)
c (P1 receive m3)	max((2,0,0),(2,3,3))=(2,3,3) → +1 own	(3,3,3)
- a (1,0,0) < e (2,2,0) ⇒ a → e ✓.		
- a (1,0,0) vs d (0,1,0): neither ≤ → **a ∥ d** (Lamport gave equal timestamps 1,1 and couldn't tell).		

Lamport vs Vector

Lamport	Vector
Single integer	Array of N integers
a→b ⇒ L(a)<L(b)	a→b ⇔ V(a)<V(b)
Cannot detect concurrency	**Detects concurrency**
O(1) space per message	O(N) space
Simple	Used for causal ordering, debugging, conflict detection (Dynamo, Riak)

🧠 Remember This

> Vector clock = one counter per process; compare component-wise; can tell **causal vs concurrent**.

---

7. SYNCHRONIZING PHYSICAL CLOCKS 📘📝 (A2-1c)

7.1 Why do physical clocks need synchronization?

- Each computer's clock is a **quartz crystal oscillator**; no two run at exactly the same rate.
- Needed for: **timestamps of transactions** (banking, e-commerce), **log/audit ordering**, **authentication tickets expiry (Kerberos)**, **distributed file timestamps (make, backup)**, **scheduling, real-time control**, **debugging**, **fair ordering of events**.
- Reference: **UTC** (Coordinated Universal Time) from atomic clocks (broadcast by GPS/radio).

7.2 Key terms (assignment-emphasized 📝)

Term	Definition	Example
Clock skew	**Difference between the readings of two clocks at the same instant**	C1=10:00:05, C2=10:00:02 → skew 3 s
Clock drift	**Rate at which a clock deviates from a perfect (reference) clock**; caused by crystal frequency variation, temperature, ageing	Typical crystal drift ≈ 10^{-6} s/s (1 µs per second ≈ **86.4 ms/day**)
Drift rate (ρ)	Max deviation per second, e.g., ρ = 10^{-6}	
Communication delay (latency)	Time taken for a message to travel; **variable** (queueing, routing) and **unknown** ⇒ received time value is already old	RTT 200 ms
Accuracy	How close to UTC	
Precision	How close clocks are to each other	

- **Relationship:** *drift causes skew to grow with time.* Two clocks with max drift ρ each can differ by up to `2ρ·Δt` after Δt.
- To keep skew ≤ δ, must resynchronise at least every **δ / 2ρ** seconds. *Example:* ρ = 10^{-6}, δ = 1 ms ⇒ resync interval ≤ 0.001/(2×10^{-6}) = **500 s**.

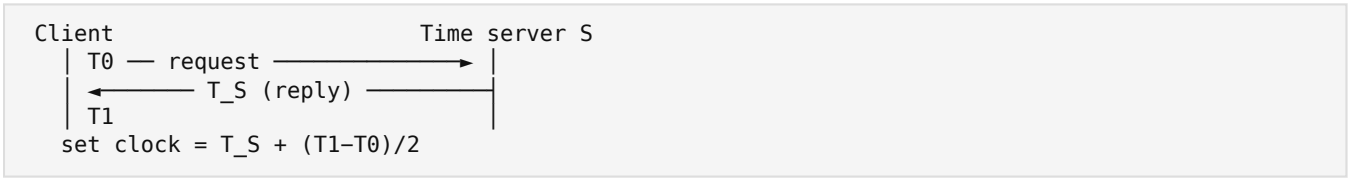
7.3 Problems/challenges with physical clocks

1. **Drift & skew** – clocks diverge continuously.
2. **Unpredictable communication delay** – time message is stale by the time it arrives; asymmetric paths.
3. **Clock cannot be set backwards** – would break time-ordering (e.g., `make`) ⇒ slow it down gradually instead.
4. **Faulty/malicious time servers** – wrong time source.
5. **No global clock** – impossible to get perfect sync.
6. **Scalability & cost** – many nodes, WAN links.
7. **Network failures** – can't reach time server.

7.4 Algorithms

(a) Cristian's Algorithm (external sync, passive time server) ➕

Purpose: sync client to a **time server** connected to a UTC source. **Assumption:** RTT is small vs required accuracy; delay roughly symmetric. **Steps:** 1. Client `p` sends request `mr` to server `S` and records **T0**. 2. S replies with its time **T_S** in message `mt`. 3. Client records reply arrival time **T1**; RTT = T1 − T0. 4. Client sets its clock to **T_S + RTT/2**. 5. Accuracy: ± (RTT/2 − min), where min = minimum one-way delay.



Numerical: T0 = 10:00:00.000, T1 = 10:00:00.800, T_S = 10:00:05.300 → RTT = 0.8 s → new time = 5.300 + 0.4 = **10:00:05.700**; accuracy ± 0.4 s (if min delay ≈ 0). **Improvements:** take several samples, use the one with **minimum RTT**. **Advantages:** simple, one message pair. **Limitations:** single point of failure (server), assumes symmetric delay, server may be faulty (fixed by using multiple servers).

(b) Berkeley Algorithm (internal sync, active master) ➕

Purpose: keep clocks of a group **mutually consistent** (no UTC source needed); average time. **Steps:** 1. **Master** (elected) polls all slaves: "what is your time?" 2. Slaves reply with their times; master compensates for RTT (Cristian-style). 3. Master computes the **average** (ignoring outliers/faulty clocks), including its own. 4. Master sends each slave the **adjustment (offset)** to apply — not the absolute time (avoids delay error). 5. Each slave adjusts its clock (gradually if it needs to go backwards).

```
        ┌──── Master 3:00 ────┐
  ask ▼                       ▼ ask
Slave1 2:50          Slave2 3:25
Master avg offset = (0 −10 +25)/3 = +5 → target 3:05
Send adjustments: Master +5, Slave1 +15, Slave2 −20
```

Numerical: as above → all clocks become **3:05**. **Advantages:** no UTC needed, tolerant to faulty clocks (discard outliers), master can be re-elected. **Limitations:** master failure/bottleneck, not very scalable, accuracy limited by RTT.

(c) NTP – Network Time Protocol ➕

- Internet-scale service; **hierarchy of servers in strata**: Stratum 0 = atomic/GPS clocks, Stratum 1 = servers directly connected, Stratum 2 syncs with Stratum 1, etc.
- Modes: **multicast** (LAN), **procedure-call** (Cristian-like), **symmetric** (highest accuracy).
- Uses timestamps `t0, t1, t2, t3` :
- **Offset θ = ((t1 − t0) + (t2 − t3)) / 2**
- **Delay δ = (t3 − t0) − (t2 − t1)**
- Pick the pair with **minimum delay**; accuracy ~ tens of ms over Internet, <1 ms LAN.
- Fault-tolerant (multiple servers, reconfigures), scalable (hierarchy), secure (authentication).

Comparison table

Feature	Cristian	Berkeley	NTP
Type	External	Internal	External (UTC)
Time source	Server with UTC	Average of group	Hierarchical servers with UTC
Master	Passive server	Active coordinator	Servers in strata
Best for	Small LAN	Group w/o UTC	Internet-wide
Fault handling	Weak	Ignores outliers	Strong

7.5 Logical Time vs Physical Time (see §8 for table).

🧠 Remember This

> **Skew** = difference between two clocks now; **drift** = rate of deviation; **delay** = uncertain message time. Cristian: T_S + RTT/2. Berkeley: average & send adjustments. NTP: stratum hierarchy.

---

8. LOGICAL TIME vs PHYSICAL TIME 📘

Basis	Physical Time	Logical Time
Meaning	Real-world time (seconds, UTC)	Counter representing **order** of events
Based on	Hardware clock/atomic clock	Events & message exchange (causality)

Basis	Physical Time	Logical Time
Purpose	Real time stamps, scheduling, timeouts	Ordering events, consistency
Synchronisation needed?	**Yes** (Cristian/Berkeley/NTP)	**No** external sync
Affected by drift/skew?	**Yes**	**No**
Accuracy	Approximate, limited by delay	Exact causal order (partial/total)
Examples	UTC, NTP time	Lamport clock, Vector clock
Relation to real time	Tied to it	Independent of it
Overhead	Periodic sync messages	Small extra timestamp in each message
Detect concurrency	No	Only vector clocks
Use case	Logs with real time, certificates	Mutual exclusion, causal broadcast, debugging

🧠 Remember This

> Physical = "kitne baje?" (needs sync). Logical = "kaun pehle?" (no clock needed).

---

9. GLOBAL STATES 📘📝 (A2-2a, A2-2b)

9.1 What is a Global State?

Definition: The **global state** of a distributed system is the **collection of the local states of all processes together with the states of all communication channels (messages in transit)** at some instant.

Simple (🗣️): Poore system ki ek "photo" – har process ki state + raaste me phansay messages.

Components of a global state

Component	Meaning
Local state of each process	Values of variables/registers of p_i
State of each channel	The set of messages sent but **not yet received** (in transit)
(Formally)	`GS = { S_1, S_2, …, S_N } ∪ { channel states C_ij }`

9.2 Cuts and consistency

- **Cut:** a subset (prefix) of the system's events – a "line" across the timelines. Frontier of the cut = last event of each process included in the cut.

- **Consistent cut:** for every event **e in the cut**, all events that **happened-before e** are also in the cut. **Equivalent:** *no message is received inside the cut whose send is outside the cut* (no **orphan message**).
- **Inconsistent cut:** contains a receive event without its send (message "from the future").
- A **consistent global state** corresponds to a consistent cut.

```
P1:  ──●──────●│──────────●──
               │  \ m1
P2:  ──●───────│──\───────●──      Cut line │:
               │   ▼                 m1 sent BEFORE cut (P1), received AFTER cut (P2) → in-transi
P3:  ──●──●────│──────────●──         → allowed; it's part of CHANNEL state.
A cut where the receive is BEFORE the line but the send is AFTER = INCONSISTENT (orphan message).
```

9.3 Why is recording a consistent global state important?

1. **Stable-property detection: deadlock detection**, **termination detection**, **garbage collection** (once true, stays true).
2. **Checkpointing & recovery:** restart from a consistent state after failure.
3. **Distributed debugging:** examine global behaviour.
4. **Auditing/consistency:** e.g., total money in a bank distributed on servers must remain constant.
5. **Load balancing, replica consistency** ➕. Problem: **no global clock and no shared memory ⇒ can't take all local states at the same instant**, so a special algorithm is required.

9.4 Distributed Snapshot – Chandy–Lamport Algorithm 📝 (A2-2b)

Purpose: record a **consistent global state** (snapshot) of a running distributed system. **Assumptions:** - Channels are **FIFO, reliable (no loss)** and **unidirectional**; processes are **connected** (strongly). - No failures during snapshot; snapshot doesn't stop the application; any process can initiate. - A special control message called **MARKER** is used.

Algorithm

```
INITIATOR P_i (or any process receiving a marker for the first time):
   1. Record own local state.
   2. Send a MARKER on EVERY outgoing channel (before any further app message).
   3. Start recording messages arriving on every incoming channel.

ON RECEIVING MARKER on incoming channel c at P_j:
   if P_j has NOT yet recorded its state:
        record local state;  channel state of c = EMPTY;
        send MARKER on all outgoing channels;
        start recording on all OTHER incoming channels.
   else (already recorded):
        stop recording channel c;
        channel state of c = messages received on c since P_j recorded its state.

TERMINATION: when every process has received a marker on ALL its incoming channels.
```

Why it works: The marker acts as a **separator** on each FIFO channel: messages before the marker belong to the "before-snapshot" world, messages after the marker to the "after".

Simple example with P1, P2, P3 (bank balances) 📝

Initial: P1 = ₹500, P2 = ₹300, P3 = ₹200 → **Total = ₹1000**. Channels: C12, C21, C13, C31, C23, C32.

Sequence of events: 1. P1 sends ₹50 to P2 (message **m_1** on C12). P1 balance = 450. 2. P1 **initiates snapshot**: records its state **S1 = 450**, sends MARKER on C12 and C13, starts recording C21 and C31. 3. P3 sends ₹20 to P1 (**m_2** on C31) before receiving any marker. P3 balance = 180. 4. P2 receives **m_1** (before marker on C12) → P2 balance = 350. Then P2 receives **MARKER on C12** → first marker: records **S2 = 350**, C12 state = {} (empty) [m_1 was received before], sends MARKERs on C21, C23, starts recording C32. 5. P3 receives **MARKER on C13** → records **S3 = 180**, C13 = {}, sends MARKERs on C31, C32, starts recording C23. 6. P1 receives m_2 on C31 **while recording** → then MARKER on C31 ⇒ **C31 = {₹20 (m_2)}**. P1 also gets MARKER on C21 → C21 = {}. 7. P2 gets marker on C32 → C32 = {}. P3 gets marker on C23 → C23 = {}. Snapshot complete.

```
P1: ──send m1──[S1=450 record, MARKER→P2,P3]──── recv m2 ── recv MARKER(C31)
P2: ──────────── recv m1 ── recv MARKER(C12)[S2=350 record, MARKER→P1,P3]
P3: ──send m2──── recv MARKER(C13)[S3=180 record, MARKER→P1,P2]
```

Recorded global state: S1 = 450, S2 = 350, S3 = 180, C31 = {₹20}, others empty. **Check:** 450 + 350 + 180 + 20 = **₹1000** ✓ (money conserved ⇒ consistent snapshot, even though no single instant ever had exactly these values).

Complexity: O(E) marker messages (one per channel, E = number of channels); at most **N(N−1)** for fully connected; O(D) time (D = diameter). **Advantages:** non-blocking (application continues), simple, works with any topology, records consistent state. **Limitations:** needs **FIFO** channels, no failures, records a state that *may never have occurred* in the actual execution (but is reachable from it), overhead of markers. **Uses:** deadlock detection, termination detection, checkpoint/recovery.

> The assignment (A2-2b) says "design a simple method for recording a consistent global state for P1, P2, P3" → answer = **Chandy–Lamport marker algorithm** with the above example.

🧠 Remember This

> Global state = process states + channel states. Consistent cut = no orphan messages. Chandy-Lamport: **marker** → record state, send markers, record incoming channels till marker.

---

10. DISTRIBUTED DEBUGGING 📘

Definition

Distributed debugging = detecting and analysing **bugs/violations of conditions (predicates)** in a running distributed program by **observing consistent global states**, since no single process can see the whole system and there is no global clock.

Simple (🗣️)

Ek computer pe debugging = breakpoint lagao. Distributed me poore system ki "sahi photo" (consistent global state) leni padti hai, tabhi bug pakadna possible hai.

Why hard?

No global clock, no global state, non-determinism (message timing), concurrency, debugging alters behaviour (**probe effect**).

Approach (Coulouris §14.5)

1. Each process **p_i** sends its **state (or changes) with vector timestamp** to a **monitor process**.
2. Monitor **assembles consistent global states** using vector timestamps (state S is consistent iff no message inconsistency between local states).
3. Monitor **evaluates a global state predicate Φ** on each consistent state.
4. Types of predicates: - **Stable predicate:** once true remains true (deadlock, termination, garbage). Detect by snapshot. - **Non-stable predicate:** can become true and false again (e.g., "x1 = x2 at some time", **safety violation**: `x − y > δ`).

Global state lattice / possibly & definitely

- The set of all consistent global states forms a **lattice**; an execution is a **path** (linearization) through the lattice from initial to final state.
- **Possibly Φ:** there **exists** a consistent global state in which Φ is true, reachable in some linearization (Φ *might* have happened).
- **Definitely Φ: every** linearization passes through a consistent global state where Φ is true (Φ *certainly* happened).
- Relations: `Definitely Φ ⇒ Possibly Φ`.

```
Initial state S00
    /      \
 S10       S01         (states labelled by #events of P1, P2)
    \  ×  /
    S11 ... → final
Φ possibly true if any path/state satisfies Φ; definitely if all paths hit such state.
```

- **Used for:** finding **safety violations**, deadlocks, **data races**, missing/duplicate messages, assertion checking; tools: logging with vector clocks, replay/record, tracing (e.g., distributed tracing).

Advantages / limitations

Advantages: correct detection of global conditions, uses causality (not time). Limitations: **state explosion** (lattice huge), high message/storage overhead, **probe effect**, needs vector timestamps.

🧠 Remember This

> Debugging = monitor collects local states with **vector timestamps** → builds **consistent global states** → tests predicate; **Possibly Φ** vs **Definitely Φ**.

---

11. COORDINATION AND AGREEMENT — INTRODUCTION 📘

- Processes must **coordinate** actions (who enters CS, who is leader, who receives which message) and **agree** on values (commit/abort, common decision) despite **failures** and **no shared memory**.
- **Failure assumptions (Coulouris §15.1):** processes may **crash** (fail-stop) or be **Byzantine**; channels reliable or not; **failure detector** (unreliable vs reliable) – in asynchronous system a timeout can't be certain.
- Topics in this section: **Distributed mutual exclusion → Election → Multicast → Consensus**.

```
COORDINATION & AGREEMENT
  ├─ Mutual exclusion (who uses resource)
  ├─ Election (who is coordinator)
  ├─ Multicast (send to group with guarantees)
  └─ Consensus (all agree on one value)
```

🧠 Remember This

> Coordination = decide who/when. Agreement = decide what. Both under failures.

---

12. DISTRIBUTED MUTUAL EXCLUSION 📘📝 (A2-3a) — ⭐ HIGH PRIORITY

Definition

Mutual exclusion (ME) ensures that **at most one process executes in its critical section (CS)** – accessing a shared resource – at a time, in a distributed system **using message passing only** (no shared memory/semaphores).

Need (🗣️)

Do process ek hi file/DB record pe ek saath likhenge to data corrupt (race condition). Isliye "ek time pe ek hi process CS me." Examples: updating a shared bank record, printing on a shared printer, writing to a file server, updating replicated database.

Requirements (Coulouris)

Requirement	Meaning
ME1 – Safety	At most **one** process in CS at a time
ME2 – Liveness	Every request to enter/exit CS **eventually succeeds** (no deadlock, no starvation)
ME3 – Ordering (→ fairness)	If request A **happened-before** request B, A is granted before B

Also: **no deadlock, no starvation, fault tolerance, fairness (FIFO order)**.

Performance criteria

1. **Bandwidth** – number of messages per CS entry/exit.
2. **Client delay** – delay at entry/exit when nobody else waiting.
3. **Synchronization delay** – time between one process leaving the CS and next entering (messages needed).

Classification

```
DISTRIBUTED MUTUAL EXCLUSION
├── Centralised (central server)
├── Token-based  : Ring algorithm, Suzuki–Kasami ➕
└── Permission-based (non-token): Ricart–Agrawala, Lamport, Maekawa (voting)
```

12.1 Algorithm 1: Central Server Algorithm

Idea: one server (coordinator) holds a **token**; process must get it to enter CS. **Steps:** 1. Process sends **request** to server. 2. If token is free → server sends **grant** immediately; else queues request. 3. After CS, process sends **release**; server grants to next in queue.

```
P1 ──request──► Server ──grant──► P1 … (CS) … P1 ──release──► Server ──grant──► P2
```

Messages: 3 per entry (request, grant, release); entry delay 2 messages; sync delay 2 messages. **Advantages:** simple, fair (queue), few messages. **Disadvantages: single point of failure**, bottleneck, needs election on server crash, can't tell dead server from long wait.

12.2 Algorithm 2: Ring-based Algorithm (token ring)

Assumption: processes arranged in a **logical ring**; each knows its neighbour; one **token** circulates. **Steps:** 1. Token passes around ring in one direction. 2. A process wanting CS **waits for token**, enters CS while holding it. 3. On exit (or if it doesn't need CS) passes token to neighbour.

```
P1 ──► P2 ──► P3
 ▲             │        token travels; only the holder may enter CS
 └─── P5 ◄── P4 ◄┘
```

Messages: 1 to N−1 per entry (avg N/2); **continuous consumption of bandwidth** even when no one wants CS. Sync delay 1 to N−1 messages. **Adv:** simple, no starvation (bounded wait ≤ N−1), deadlock-free. **Disadv: token loss** (needs detection/regeneration), wasted messages, process crash breaks ring, ordering not FIFO/happened-before.

12.3 Algorithm 3: Ricart–Agrawala Algorithm (multicast + logical clocks) — ⭐ MOST ASKED

Purpose: ME **without a token or central server**, using **Lamport timestamps** for a total order of requests. **Assumptions:** N processes; reliable message delivery; each process has unique id and Lamport clock; processes know each other; no failures. **States:** RELEASED (not interested), WANTED (wants CS), HELD (in CS).

Algorithm

```
On initialisation:   state := RELEASED

To ENTER the CS:
   state := WANTED
   multicast REQUEST(T, p_i)  to all other processes     (T = timestamp of request)
   wait until N-1 REPLIES received
   state := HELD

On RECEIPT of REQUEST(T_j, p_j) at p_i:
   if ( state == HELD  or  (state == WANTED and (T_i, p_i) < (T_j, p_j)) )
        queue the request (defer reply)
   else
        reply to p_j immediately

To EXIT the CS:
   state := RELEASED
   send REPLY to all queued requests
```

(Comparison (T,p) uses **smaller timestamp = higher priority; tie broken by process id**.)

Example (3 processes P1, P2, P3): P1 and P2 request simultaneously with timestamps (5,P1) and (7,P2). P3 is RELEASED. 1. P1 → REQUEST(5,P1) to P2, P3; P2 → REQUEST(7,P2) to P1, P3. 2. P3 (RELEASED) replies to both immediately. 3. P1 receives P2's request: P1 is WANTED with (5,P1) < (7,P2) ⇒ **defers** (queues) P2's reply. 4. P2 receives P1's request: P2 is WANTED with (7,P2) > (5,P1) ⇒ **replies** to P1. 5. P1 has replies from P2 and P3 ⇒ **enters CS**. 6. On exit P1 sends the deferred REPLY to P2 ⇒ P2 has replies from P1 and P3 ⇒ **enters CS**.

```
P1 ──REQ(5)──► P2, P3          P2 ──REQ(7)──► P1, P3
P3 ── REPLY ─► P1, P2
P2 ── REPLY ─► P1  → P1 enters CS
P1 (exit) ── REPLY ─► P2 → P2 enters CS
```

Message complexity: 2(N−1) per CS entry: (N−1) requests + (N−1) replies. **Synchronization delay:** 1 message transmission time. **Client delay:** round-trip. **Advantages:** no token/coordinator, satisfies ME1–ME3 (fair, FIFO by timestamps), deadlock- & starvation-free, no release messages. **Disadvantages: N−1 messages each entry ⇒ poor**

scalability, failure of **any** process blocks everyone (no reply), every process involved even if uninterested, needs reliable delivery. (Optimisation: **Maekawa's** voting sets.)

12.4 Algorithm 4: Lamport's Distributed Mutual Exclusion ➕ (also in your lab list)

Idea: each site keeps a **request queue ordered by (timestamp, id)**. **Steps:** 1. To request CS, P_i sends **REQUEST(ts_i, i)** to all others and puts it in its own queue. 2. On receiving a request, P_j puts it in its queue and sends **REPLY** (timestamped). 3. P_i **enters CS** when (a) its request is **at the head of its queue**, and (b) it has received a **reply with a larger timestamp from every other process**. 4. On exit, P_i removes its request and sends **RELEASE** to all; others remove that request from their queue. **Messages: 3(N−1)** per CS (N−1 REQUEST + N−1 REPLY + N−1 RELEASE). Sync delay = 1 message. Requires **FIFO channels**.

12.5 Algorithm 5: Maekawa's Voting Algorithm ➕

Idea: each process needs permission only from a **voting set V_i** (subset of size ≈ √N) such that every two voting sets overlap ($V_i \cap V_j \neq \emptyset$). A process can vote for only one candidate at a time. **Steps:** request → send to all members of V_i; a member replies (votes) if it hasn't voted since the last release; otherwise queues; enter CS after getting **all** votes; on exit send **release** to V_i. **Messages:** ≈ **3√N** (2√N entry + √N exit). **Problem: deadlock-prone** (fixed by ordering requests by happened-before).

12.6 Comparison table – mutual exclusion algorithms (⭐ exam)

Algorithm	Type	Messages per entry	Sync delay (message times)	Problems	Remarks
Central server	Centralised	**3** (req, grant, release)	2	Server crash = SPOF	Simple, fair
Token ring	Token	**1 to N−1** (continuous ring traffic)	1 to N−1	Token loss, node crash	No starvation
Ricart–Agrawala	Permission	**2(N−1)**	1	Any crash blocks; heavy	Timestamps, no release msg
Lamport	Permission	**3(N−1)**	1	Needs FIFO; heavy	Uses request queues
Maekawa	Voting	**~3√N**	2	Deadlock possible	Scalable
Suzuki–Kasami ➕	Broadcast token	**N** (N−1 requests + 1 token) or 0 if token held	1	Token loss	Efficient

Numerical (N = 5): RA = 2×4 = **8**; Lamport = 3×4 = **12**; Central = **3**; Ring = 1–4; Maekawa ≈ 3√5 ≈ 7 (roughly).

12.7 Exam points

- ⭐ For a 4-mark answer "requirements + any two algorithms": ME1, ME2, ME3 (1 mark) + **Central server + Ricart–Agrawala** (or Ring) with diagram and message count (3 marks).
- Be ready to say how each **fails** (crash, token loss) and which is best in message count (central server) vs fully distributed (RA).

🧠 Remember This

> ME1 safety, ME2 liveness, ME3 ordering. Central: 3 msgs. Ring: token circulates. **Ricart-Agrawala: 2(N−1), timestamp comparison, defer replies while HELD/higher priority.** Lamport: 3(N−1).

---

13. ELECTION ALGORITHMS 📘📝 (A2-3b) — ⭐ HIGH PRIORITY

Definition

An **election algorithm** chooses **one process as coordinator (leader)** from a group of processes (e.g., the process with the **highest ID**) so that all processes agree on who it is.

Need for election (🗣️: "leader kaun?")

- A **coordinator** is needed for central-server mutual exclusion, time server (Berkeley master), primary in replication, token regeneration, etc.
- **When the coordinator crashes** or at system start, the remaining processes must **elect a new one**.
- Requirements: **E1 Safety** – every participating process has elected the same process (the non-crashed one with the **largest identifier**); **E2 Liveness** – all processes participate and eventually set elected variable or crash.
- Assumption: every process has a **unique ID** (priority) and knows the IDs of others (bully) or its neighbour (ring). Message types in Bully: **ELECTION, ANSWER (OK), COORDINATOR**.

13.1 Bully Algorithm (Garcia-Molina)

Idea: The process with the **highest ID** always wins ("bullies" others). **Assumptions: synchronous** system (timeouts reliable); reliable message delivery; each process knows the IDs of **all** others; failures only crash; crashed process may recover.

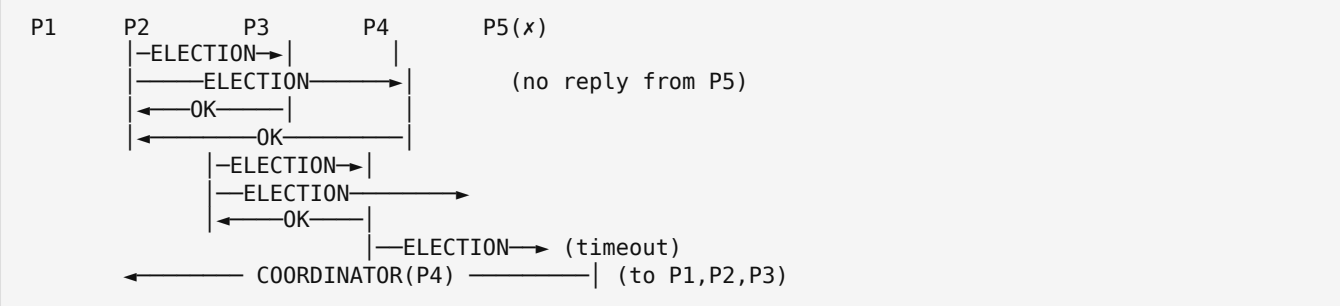
Steps 1. A process P that **notices coordinator failure** starts an **election**: sends **ELECTION** to all processes with **higher ID**. 2. If **no one responds** (timeout) → P **wins**; it sends **COORDINATOR** message to all lower-ID processes (in fact to all). 3. If a **higher process responds with ANSWER (OK)** → P's job is over; it waits for a COORDINATOR message. 4. Each higher process that receives ELECTION replies **OK** and **starts its own election** (step 1).

5. Eventually the **highest alive process** gets no answer, and announces **COORDINATOR**. 6. If a crashed process **recovers**, it starts an election; if it has the highest ID it takes over ("bully").

Example (5 processes P1–P5, P5 was coordinator and crashes ✗):

```
Step 1: P2 detects P5 down → sends ELECTION to P3, P4, P5.
Step 2: P3 and P4 reply OK to P2. (P5 silent ✗)  P2 stops.
Step 3: P3 sends ELECTION to P4, P5.   P4 sends ELECTION to P5.
Step 4: P4 replies OK to P3. P5 silent. P3 stops.
Step 5: P4 timeouts waiting for P5 → P4 wins → sends COORDINATOR(4) to P1,P2,P3.
```

Message flow diagram:



Message complexity: Best case: the process with the second-highest ID detects failure → it sends **N−2** COORDINATOR messages immediately. **Worst case:** the process with the **lowest ID** detects failure → **O(N²)** messages (all higher processes start elections). **Advantages:** simple, fast when high-ID processes are alive, tolerates recovery; guarantees highest-ID wins. **Disadvantages: many messages O(N²)** worst case, requires **timeouts (synchronous)**, every process must know all IDs, if high-priority process flaps (crashes/recovers repeatedly) it causes repeated elections, incorrect timeouts can cause two coordinators.

13.2 Ring Algorithm (Chang–Roberts)

Idea: processes arranged in a **logical ring** (each knows its successor); an **election message** circulates carrying the **highest ID seen** so far; the process that receives **its own ID back** is the winner. **Assumptions:** ring topology, **unidirectional**; each process knows its **next neighbour** (skips crashed); works in an **asynchronous** system; unique IDs; no failure during election.

Steps 1. Initially every process is **non-participant**. A process P noticing failure marks itself **participant** and sends **ELECTION(id_P)** to its neighbour. 2. On receiving ELECTION(id) at Q: - if **id > id_Q**: forward ELECTION(id); mark Q participant. - if **id < id_Q** and Q is **not participant**: **substitute** own id, send ELECTION(id_Q), mark participant. - if **id < id_Q** and Q **already participant**: **discard** (don't forward). - if **id == id_Q**: Q is the **coordinator**; it marks itself non-participant and sends **ELECTED(id_Q)** around the ring. 3. When ELECTED message comes to a process, it records the coordinator, marks itself non-participant and forwards; it stops when it returns to the coordinator.

Example: ring P1→P2→P3→P4→P5→P1 with P5 crashed (skip it): P4→P1. P2 starts election with id 2.

```
P2 —ELECTION(2)► P3 (3>2 → substitute 3) —ELECTION(3)► P4 (4>3 → substitute 4) —ELECTION(4)► P1(1<4
—ELECTION(4)► P2 (2<4 → forward) —► P3 (forward) —► P4 receives its own id 4 → P4 = COORDINATOR
P4 —ELECTED(4)► P1 —► P2 —► P3 —► back to P4 (end)
```

```
      P1
     ↗  ↘
  P4      P2        election msg circulates clockwise carrying highest id;
     ↖  ↙           P4 (highest alive) receives its own id → sends ELECTED(4)
      P3            (P5 crashed and skipped)
```

Message complexity: Best: 2N (election N + elected N) — when the initiator has highest id. **Worst: 3N − 1** (when the initiator's **anticlockwise neighbour** is the highest: N−1 messages to reach it, N to complete the circle, N for elected). **Advantages:** fewer messages than bully, no need to know all IDs, works in asynchronous systems, simple ordering. **Disadvantages:** slow for large rings (O(N) latency), a **failure of a node during election breaks the ring**, needs ring reconstruction, several simultaneous elections possible (handled by participant flag).

13.3 Comparison: Bully vs Ring (📝 A2-3b: "compare their message communication and assumptions")

Feature	Bully Algorithm	Ring Algorithm
Topology	**Fully connected** (any process can message any other)	**Logical ring** (only to neighbour)
Knowledge needed	IDs of **all** processes	Only **next neighbour**
Messages used	ELECTION, ANSWER/OK, COORDINATOR	ELECTION (with id), ELECTED
Best-case messages	**N−2**	**2N**
Worst-case messages	**O(N²)**	**3N−1** = O(N)
System assumption	**Synchronous** (timeouts)	**Asynchronous** OK
Winner	Highest-ID alive process	Highest-ID alive process
Who announces	Winner sends COORDINATOR to all	ELECTED message circulates ring
Failure during election	Handled via timeouts, may restart	Ring broken; needs repair
Speed	Fast when initiator has high id	Slower (message travels whole ring)
Simultaneous elections	Multiple starters possible	Handled by participant flag
Recovery of process	Recovered process may start election / bully	Rejoins ring, may trigger election

Exam points

- Draw a 5-process example for each algorithm; state message types; give best/worst counts; **the key line: "highest ID wins in both"**.

🧠 Remember This

> **Bully:** ELECTION→higher ids; OK; highest alive sends COORDINATOR; worst $O(N^2)$. **Ring:** id circulates, own id returns → winner; ELECTED circulates; worst $3N-1$.

---

14. MULTICAST COMMUNICATION 📘

Definition

Multicast = sending **one message to all members of a group** of processes (one-to-many), as opposed to unicast (one-to-one) and broadcast (to everyone).

Simple (🗣️)

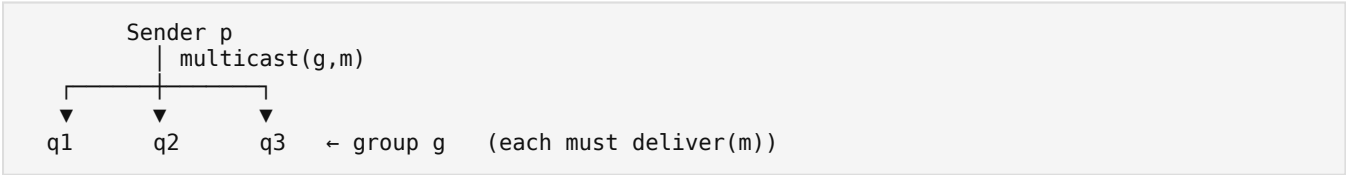
WhatsApp group me ek message sabhi members ko jaata hai. Yahi multicast hai – par DS me guarantee chahiye: sabko mile, ek order me mile.

Uses (why needed)

Fault tolerance via **replicated services**, **finding discovery services** in spontaneous networking, **better performance** through replicated data, **event notification/propagation** (stock prices, news), online games, video conferencing, cache consistency.

Group system model

- **Closed group:** only members can multicast (replicated servers). **Open group:** outsiders can send too (event dissemination).
- **Overlapping groups** possible; **synchronous/asynchronous** systems.
- Operations: `multicast(g, m)`, `deliver(m)` (delivered to application ≠ received on the network; **hold-back queue** may delay delivery).
- **IP multicast** (➕): efficient, unreliable UDP-based; class D addresses; routers replicate packets.



14.1 Basic (B-)multicast

`B-multicast(g,m)`: for each q in g: `send(q, m)` (over reliable 1-to-1 channels). `B-deliver(m)` on receive. **Problem:** if sender crashes midway, some members get m and some don't ⇒ no agreement. (Also **ack-implosion** if all ack.)

14.2 Reliable multicast – properties (Coulouris)

Property	Meaning
Integrity	A correct process delivers a message **at most once**, and only if sent by a group member
Validity	If a correct process multicasts m, it **eventually delivers** m itself
Agreement	If a correct process delivers m, then **all other correct processes** in the group eventually deliver m
- **Reliable multicast R-multicast** built over B-multicast: when a process first receives message m, if it is not the sender it **re-multicasts (B-multicasts) m to the group** before delivering; ensures agreement even if sender crashes. Costly ($O(N^2)$ messages). Alternatively use **piggybacked acknowledgements** and **negative acks (NACK)** for efficiency.	
- **Uniform agreement** (➕): if *any* process (even one that later crashes) delivers m, all correct processes do.	

14.3 Ordered multicast

Ordering	Guarantee	Typical implementation
FIFO ordering	If a process multicasts m1 before m2, every correct process delivers m1 before m2	**Sequence numbers** per sender, hold-back queue
Causal ordering	If multicast(g,m1) → multicast(g,m2) (happened-before) then every process delivers m1 before m2	**Vector timestamps**
Total ordering	If a correct process delivers m1 before m2, **all** correct processes deliver m1 before m2 (same order everywhere)	**Sequencer process** (assigns sequence numbers) or **ISIS agreed-priority** algorithm
- Hierarchy: **Causal ⊃ FIFO** (causal implies FIFO); Total is independent; **FIFO-total, causal-total** combinations exist.		

Ordering	Guarantee	Typical implementation
- **Total ordering with a sequencer:** sender sends m to sequencer; sequencer assigns sequence number s and multicasts (m,s); receivers deliver in order of s (hold-back until earlier numbers arrive).		
- **ISIS algorithm:** sender B-multicasts m; each receiver replies with a **proposed priority**; sender picks the **maximum** as **agreed priority** and multicasts it; receivers deliver in agreed-priority order.		

```
Total order example: P1 sends m1, P2 sends m2 concurrently.
All processes must deliver in the same order: either (m1,m2) everywhere or (m2,m1) everywhere.
```

Hold-back queue (diagram):

```
Network ─► [Hold-back queue] ─(when ordering condition met)─► [Delivery queue] ─► Application
```

Advantages: efficient dissemination, replication support, less network traffic than N unicasts.
Disadvantages: ordering/reliability complexity, message overhead (acks), scalability, failure handling, group membership management.

Exam points

- Draw group diagram; define reliable multicast (integrity, validity, agreement); compare FIFO/causal/total ordering with one example.

🧠 Remember This

> Multicast = one→group. Reliable: **integrity, validity, agreement**. Ordered: **FIFO, causal (vector clocks), total (sequencer/ISIS)**.

---

15. CONSENSUS AND RELATED PROBLEMS 📘📝 (A2-4a, A2-4b) — ⭐ HIGH PRIORITY

Definition

Consensus: a set of processes each **proposes a value**; they must all **agree on one value** (decide), even if some processes fail.

Simple (🗣️)

5 dost movie choose karte hain – har koi apni pasand batata hai, aakhir me **sabko ek hi movie** par agree hona hai, chahe ek dost phone band kar de.

Need for consensus

Commit or abort a **transaction** (2PC), **leader election**, **replicated state machines**/replicated database consistency, **agreeing on order of messages** (total-order multicast), **mutual exclusion / access decision**, blockchain, **agreeing on time or sensor value** in fault-tolerant control, **airplane control systems** (redundant computers).

Requirements (Coulouris §15.5) 📝

Each process p_i starts in **undecided** state, proposes value v_i, then **decides** value d_i (decided state, cannot change). | Condition | Meaning | |---|---| | **C1 – Termination** | Every correct process **eventually decides** (sets its decision variable) | | **C2 – Agreement** | The decision value of **all correct processes is the same**: if p_i and p_j are correct and decided, then d_i = d_j | | **C3 – Integrity / Validity** | If all correct processes proposed the **same value v**, then any correct process that decides has decided **v** (decided value must be one that was **proposed**; not arbitrary) |

> Your syllabus/assignment wording: **Agreement, Validity, Termination** – same three conditions.

Simple example (assignment 4a)

Three replicas of a database decide whether to **commit** a transaction. Proposals: P1 = commit, P2 = commit, P3 = abort. Using a protocol (e.g., majority): decision = **commit** by all. Agreement ✓ (all commit), validity ✓ (commit was proposed), termination ✓ (all decided). If all proposed commit, decision must be commit (validity).



Consensus in a synchronous system with crash failures – flooding algorithm ➕

- Up to **f** processes may crash; **f + 1 rounds** needed.
- Each process keeps set `Values_i`. In each round, it **B-multicasts** the values it hasn't sent yet, then adds received values.
- After **f+1 rounds**, each process decides on **minimum (or majority function) of Values_i**.
- Why f+1: at least one round has **no crash**, ensuring all correct processes have the same set.

```
Round 1: each Pi multicasts its value; crash may cause partial delivery
Round 2..f+1: multicast newly learned values
Decide: d = min(Values_i)
```

Complexity: (f+1) rounds; O(N²) messages per round.

Process failures & communication delays (assignment 4b challenges) 📝

Challenge	Effect
Crash failure	A process stops silently; others can't distinguish crash from slowness in asynchronous systems
Byzantine (arbitrary) failure	Process sends **conflicting/wrong values** to different peers
Message loss / omission	Some messages never arrive → incomplete view
Communication delay (async)	Unbounded delays make timeouts unreliable; process may be wrongly declared dead
Partial multicast	Failed process sends its value to only some processes → disagreement
Termination vs agreement conflict	Waiting for failed process ⇒ never terminates
- **FLP impossibility (Fischer–Lynch–Paterson, 1985):** in a **purely asynchronous** system, **no deterministic consensus algorithm** can guarantee termination if even **one** process may crash. Practical systems use **timeouts, randomisation, failure detectors** (Paxos, Raft).	

🎯 Case study 4b: 5 processes, at most 1 may fail

Case 1 – Crash failure (f = 1): N = 5, use **f+1 = 2 rounds**. 1. Round 1: each process broadcasts its proposed value to all. 2. Suppose P5 crashes after sending its value to only P1 and P2. 3. Round 2: processes re-broadcast any new values they learnt (P1, P2 forward P5's value to P3, P4). 4. After round 2 all four **correct** processes have the **same set**. Each applies the same deterministic rule (e.g., **minimum** or **majority**) ⇒ **agreement**. 5. Validity: chosen value was proposed; Termination: decision after 2 rounds.

Case 2 – Byzantine failure (f = 1): need **N ≥ 3f + 1 = 4** processes (we have 5 ✓) and **f+1 = 2 rounds**. Each process collects values; the faulty one may send different values to different peers; correct ones **exchange what they received** and take **majority** ⇒ faulty process's lies are outvoted by the 4 correct processes. **Challenges:** failure detection uncertain, delays cause timeouts, message overhead grows (O(N²) per round), need reliable channels, need common decision rule.

Consensus-related problems 📘 ("consensus and related problems")

Problem	Description	Condition
Byzantine Generals Problem	Commander sends order to lieutenants; some (possibly the commander) may be traitors. Loyal lieutenants must **agree** (BG1) and if commander is loyal, they follow his order (BG2 – integrity)	Solvable only if **N ≥ 3f + 1** (oral messages); with signed messages any number tolerated (in sync systems)
Interactive Consistency	Each process proposes value; all must agree on a **vector of values** (one per process)	IC1 agreement on vector; IC2 if p_i correct, everyone's entry for p_i is v_i
Total-ordered multicast	All deliver messages in same order (equivalent to consensus)	Solves & is solved by consensus
Leader election	Agree on one leader (consensus on id)	
Atomic commit (2PC/3PC) ➕	All participants commit or all abort	
Reliable multicast	Agreement on delivered messages	
Relationships: Consensus (C), Byzantine Generals (BG), Interactive Consistency (IC) can each be **derived from** the others, so they are **equivalent** (Coulouris).		

Byzantine tolerance bound: with **f** faulty processes need **N ≥ 3f+1**; **f+1 rounds** (synchronous). Example: **N=3, f=1 impossible** – loyal process can't tell whether commander or another lieutenant is lying.

```
N=4, f=1: commander C loyal sends v to L1,L2,L3; L3 faulty tells others "commander said w".
L1 majority(v, v, w) = v; L2 majority = v → agreement.
```

Advantages/limitations of consensus protocols

Advantages: fault-tolerant, consistent replicas. Limitations: high message cost, FLP impossibility, complex, requires majority/quorum. **Practical protocols ➕:** Paxos, Raft (majority quorum, tolerate f crash failures with 2f+1 nodes).

🧠 Remember This

> Consensus = agree on one value. **Termination + Agreement + Validity(Integrity)**. Crash: f+1 rounds. Byzantine: N ≥ 3f+1. Async + 1 crash ⇒ no deterministic guarantee (FLP).

(End of Unit-II. Next: `03_Faculty_Emphasis_and_Quick_Tables.md`)